

GUIDE MÉTHODOLOGIQUE UNIVERSEL : CTP SQL

Framework de résolution pas-à-pas applicable à toutes les variantes de l'examen (Logistique & Transport)

Note de structure : Les deux sujets de CTP (Sujet 1 : Plateforme Logistique / Ventes et Sujet 2 : Gestion des Colis et Transporteurs) possèdent une architecture **identique en 7 étapes**. Ce guide fournit une méthodologie systématique et met en parallèle le code exact pour les deux sujets afin de garantir une réussite maximale sans aucune confusion sur les noms d'entités.

| Règle d'Or Absolue du CTP

- **Zéro théorie demandée** : Seul le résultat affiché à l'écran valide les points.
- **Validation séquentielle** : Vous devez impérativement montrer le résultat de l'étape en cours à l'enseignant avant de saisir les commandes de l'étape suivante.
- **Gestion des dépendances** : Créez toujours les tables indépendantes (sans clés étrangères) avant les tables dépendantes pour éviter les erreurs de référence.

| Étapes 1 & 2 — Création des Tables et Relations (DDL)

L'objectif est de traduire le diagramme fourni en tables SQL viables. Pour interdire logiquement qu'un élément "enfant" existe sans son "parent" (ex: une commande sans client, ou un colis sans transporteur), appliquez la contrainte NOT NULL sur la clé étrangère.

```
-- 1. Tables indépendantes
CREATE TABLE client (
    id_client INT PRIMARY KEY,
    nom VARCHAR(100),
    ville VARCHAR(100)
);

CREATE TABLE article (
    id_article INT PRIMARY KEY,
    designation VARCHAR(150),
    prix DECIMAL(10,2)
);

-- 2. Tables dépendantes
CREATE TABLE commande (
    id_commande INT PRIMARY KEY,
    date_commande DATE,
    mode_paiement VARCHAR(50),
    id_client INT NOT NULL,
    FOREIGN KEY (id_client)
        REFERENCES client(id_client)
);

-- Table d'association (Relation N-N)
CREATE TABLE contient (
    id_commande INT,
    id_article INT,
    quantite INT NOT NULL,
    PRIMARY KEY (id_commande, id_article),
    FOREIGN KEY (id_commande)
        REFERENCES commande(id_commande),
    FOREIGN KEY (id_article)
        REFERENCES article(id_article)
);
```

```
-- 1. Tables indépendantes
CREATE TABLE transporteur (
    id_transporteur INT PRIMARY KEY,
    nom VARCHAR(100),
    ville VARCHAR(100)
);

CREATE TABLE type_carton (
    id_type_carton INT PRIMARY KEY,
    format_nom VARCHAR(50),
    hauteur DECIMAL(5,2)
);

CREATE TABLE casier (
    id_casier INT PRIMARY KEY,
    zone_stockage VARCHAR(50)
);

-- 2. Table dépendante
CREATE TABLE colis (
    ref_colis VARCHAR(50) PRIMARY KEY,
    date_preparation DATE NULL,
    id_transporteur INT NOT NULL,
    id_type_carton INT NOT NULL,
    FOREIGN KEY (id_transporteur)
        REFERENCES
            transporteur(id_transporteur),
    FOREIGN KEY (id_type_carton)
        REFERENCES
            type_carton(id_type_carton)
);
```

Étape 3 — Insertion des Données (DML)

Stratégie d'insertion : Vous devez impérativement insérer les lignes dans les tables "mères" avant d'insérer des données dans les tables "filles" contenant des clés étrangères, sous peine de déclencher une erreur de violation de contrainte d'intégrité (Foreign Key Violation).

Sujet 1 — Jeu d'essai (4 Clients, 5 Articles, 6 Commandes, 8 Lignes "Contient")

```
INSERT INTO client VALUES
(1, 'Dupont Jean', 'Lille'), (2, 'Martin Sophie', 'Paris'), (3, 'Bernard Luc', 'Lille'), (4,
'Dubois Marie', 'Lyon');

INSERT INTO article VALUES
(1, 'Ordinateur', 850.00), (2, 'Souris', 25.00), (3, 'Clavier', 75.00), (4, 'Écran', 200.00),
(5, 'Câble HDMI', 10.00);

INSERT INTO commande VALUES
(101, '2026-05-20', 'CB', 1), (102, '2026-05-21', 'Paypal', 2), (103, '2026-05-22', 'CB', 1),
(104, '2026-05-22', 'Virement', 3), (105, '2026-05-23', 'CB', 4), (106, '2026-05-24', 'Chèque',
2);

INSERT INTO contient VALUES
(101, 1, 1), (101, 2, 2), (102, 4, 2), (103, 5, 4), (104, 5, 3), (105, 3, 1), (106, 2, 1),
(106, 1, 1);
```

Sujet 2 — Jeu d'essai (3 Transporteurs, 3 Cartons, 4 Casiers, 5 Colis)

```
INSERT INTO transporteur VALUES
(1, 'DHL France', 'Lille'), (2, 'FedEx Europe', 'Paris'), (3, 'DPD Logistics', 'Lille');

INSERT INTO type_carton VALUES
(1, 'Petit Format', 15.50), (2, 'Format Moyen', 30.00), (3, 'Grand Format', 60.00);

INSERT INTO casier VALUES
(10, 'Zone A-Sud'), (20, 'Zone A-Nord'), (30, 'Zone B-Est'), (40, 'Zone B-Ouest');

-- Notez l'insertion délibérée de valeurs NULL pour la date de préparation afin de valider
l'étape 4
INSERT INTO colis VALUES
('COL-001', '2026-05-24', 1, 1),
('COL-002', NULL, 3, 2),
('COL-003', '2026-05-25', 1, 3),
('COL-004', NULL, 2, 2),
('COL-005', '2026-05-22', 3, 1);
```

Étape 4 — Requêtes de Filtrage Simples

Cette étape teste votre maîtrise des clauses de restriction de base de l'élément WHERE : les égalités basiques, la recherche de motifs textuels (LIKE avec le joker %), et la détection des valeurs manquantes (IS NULL).

Type de filtre recherché	SUJET 1 Code & Objectif	SUJET 2 Code & Objectif
Filtre de texte précis (Égalité stricte)	<i>Clients habitant à Lille :</i> <pre>SELECT * FROM client WHERE ville = 'Lille';</pre>	<i>Transporteurs basés à Lille :</i> <pre>SELECT * FROM transporteur WHERE ville = 'Lille';</pre>
Filtre sur valeur absente (Marqueur NULL)	<i>(Cas théorique non demandé au sujet 1)</i> <pre>SELECT * FROM commande WHERE date_commande IS NULL;</pre>	<i>Colis dont la date de préparation est manquante :</i> <pre>SELECT * FROM colis WHERE date_preparation IS NULL;</pre>
Recherche partielle / Motif (Joker textuel)	<i>Articles triés par prix (Décroissant) :</i> <pre>SELECT * FROM article ORDER BY prix DESC;</pre>	<i>Transporteurs dont le nom commence par un "D" :</i> <pre>SELECT * FROM transporteur WHERE nom LIKE 'D%';</pre>

Étape 5 — Les Jointures Relationnelles (INNER JOIN)

La règle fondamentale de la jointure est de lier la clé primaire (PK) d'une table avec la clé étrangère (FK) de la table associée à l'intérieur du critère de raccordement ON. Préfixez systématiquement vos colonnes avec le nom de la table pour éliminer toute ambiguïté de lecture pour le SGBD.

Sujet 1 — Jointures demandées

1. Afficher le nom du client associé à la date de sa commande :

```
SELECT client.nom, commande.date_commande
FROM client
INNER JOIN commande ON client.id_client = commande.id_client;
```

2. Afficher la désignation de l'article associée à sa quantité commandée :

```
SELECT article.designation, contient.quantite
FROM article
INNER JOIN contient ON article.id_article = contient.id_article;
```

Sujet 2 — Jointures demandées

1. Afficher la référence du colis associée au nom de son transporteur :

```
SELECT colis.ref_colis, transporteur.nom AS nom_transporteur
FROM colis
INNER JOIN transporteur ON colis.id_transporteur = transporteur.id_transporteur;
```

2. Afficher la référence du colis associée au type de son carton d'emballage :

```
SELECT colis.ref_colis, type_carton.format_nom
FROM colis
INNER JOIN type_carton ON colis.id_type_carton = type_carton.id_type_carton;
```

Étape 6 — Calculs et Fonctions d'Agrégation

Règle d'or du GROUP BY : Toutes les colonnes présentes dans la clause SELECT qui ne font pas l'objet d'une fonction d'agrégation (telle que SUM, COUNT, AVG) doivent obligatoirement figurer dans la clause GROUP BY. Pour filtrer un résultat calculé par un agrégat, le mot-clé WHERE est interdit ; vous devez impérativement utiliser la clause HAVING après le regroupement.

1. Total des quantités par article :

```
SELECT article.designation,  
       SUM(contient.quantite) AS total  
FROM article  
JOIN contient  
  ON article.id_article =  
  contient.id_article  
GROUP BY article.designation;
```

2. Uniquement les totaux > 5 (HAVING) :

```
SELECT article.designation,  
       SUM(contient.quantite) AS total  
FROM article  
JOIN contient  
  ON article.id_article =  
  contient.id_article  
GROUP BY article.designation  
HAVING SUM(contient.quantite) > 5;
```

1. Nombre de colis attribués par transporteur :

```
SELECT transporteur.nom,  
       COUNT(colis.ref_colis) AS nb_colis  
FROM transporteur  
LEFT JOIN colis  
  ON transporteur.id_transporteur =  
  colis.id_transporteur  
GROUP BY transporteur.nom;
```

2. Moyenne arithmétique de la hauteur des cartons :

```
SELECT AVG(type_carton.hauteur) AS  
moyenne_hauteur  
FROM type_carton;
```

Étape 7 — Requêtes Avancées (Sous-requêtes ou Fonctions Window)

L'enseignant vous offre le choix d'utiliser soit une approche classique par **sous-requête** (idéale pour isoler un extremum absolu via `MAX()`), soit une approche moderne par **fonction de fenêtrage (Window Function)** (idéale pour générer des classements complets sans effondrer la structure des lignes).

Option A : L'approche par Sous-Requête (Isoler la valeur maximale)

On cherche à afficher la ligne contenant la valeur maximale (le prix le plus élevé ou la date la plus récente) en effectuant une comparaison dynamique avec le résultat isolé d'un sous-bloc.

SUJET 1 *Trouver l'article le plus cher du catalogue :*

```
SELECT designation, prix
FROM article
WHERE prix = (SELECT MAX(prix) FROM article);
```

SUJET 2 *Trouver le colis préparé le plus récemment (Date maximale) :*

```
SELECT ref_colis, date_preparation
FROM colis
WHERE date_preparation = (SELECT MAX(date_preparation) FROM colis);
```

Option B : L'approche par Fonction Window (Établir un classement analytique)

La clause `OVER (ORDER BY ...)` permet de calculer le rang de chaque ligne par rapport aux autres sans avoir besoin de regrouper les lignes du résultat final.

SUJET 1 *Générer le classement complet des articles selon leur prix décroissant :*

```
SELECT designation, prix,
       RANK() OVER (ORDER BY prix DESC) AS rang_prix
FROM article;
```

SUJET 2 *Générer le classement complet des colis du plus récent au plus ancien :*

```
SELECT ref_colis, date_preparation,
       RANK() OVER (ORDER BY date_preparation DESC) AS rang_recence
FROM colis
WHERE date_preparation IS NOT NULL;
```

MÉMENTO EXAMEN : Gardez ce guide ouvert ou mémorisez cette progression linéaire. Les types de fonctions et l'ordre logique d'exécution SQL (`FROM` → `JOIN` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY`) restent strictement immuables d'un sujet à l'autre.